ResNet5M: A ResNet Under 5 Million Parameters for CIFAR-10

Harsh Kumar Sah (hs6259) and Maaz Imam (smi6079)
NYU Tandon School of Engineering, Deep Learning, Fall 2026, Project 1
Kaggle team: resnet
Code: https://github.com/thisisharshsah/resnet5m

1. Overview

We trained a ResNet from scratch to classify CIFAR-10 images with fewer than 5 million trainable parameters, as the course competition requires. Our final model has 4,902,506 parameters. It reaches 97.46% accuracy on our own 5,000 image validation set and 0.899 on the Kaggle public leaderboard.

CIFAR-10 [15] has 50,000 labelled training images of 32x32 pixels in 10 classes. The competition scores predictions on 2,000 hidden test images: half of them form the public leaderboard and the other half the private one. The rules ask for a ResNet trained from scratch, with no pretrained weights, no outside data and at most 5,000,000 trainable parameters in the submitted model.

Three ideas gave most of our improvement:

1. **Wide and shallow.** With a fixed parameter budget, three wide stages worked better than the deeper four stage ResNet-18 layout, in line with wide ResNets [2].
2. **Strong regularization.** Heavy data augmentation, sharpness aware minimization (SAM) and an average of the weights over time (EMA) let the model train for 1,600 epochs without overfitting.
3. **Learning from bigger models.** Five larger ResNets (36.7 million parameters each, also trained from scratch on the same training images) act as teachers, and our small model learns to copy their averaged predictions. This is called knowledge distillation [3]. Only the small model is submitted.

The rest of this report covers the model (Section 2), the training method (Section 3), distillation (Section 4), our experiments (Section 5), the results (Section 6) and what we learned (Section 7).

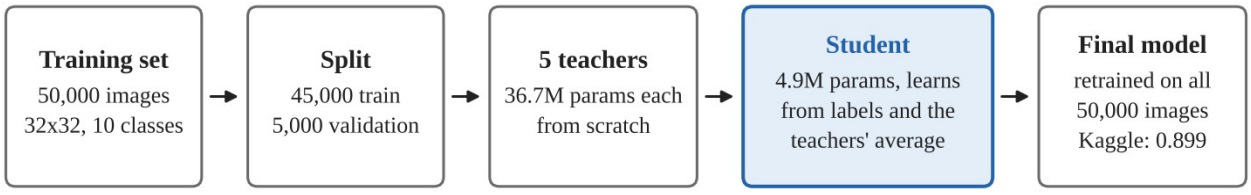


Figure 1. *Our pipeline. The teachers are used only during training, and only the student is submitted.*

2. Model Architecture

Our model is a standard ResNet [1, 14] with three stages and two residual blocks per stage (Figure 2). Each residual block computes

$$y = ReLU(F(x) + S(x)), \quad F = conv\ 3x3 \rightarrow BN \rightarrow ReLU \rightarrow conv\ 3x3 \rightarrow BN$$

where S is the shortcut: the input itself, or a 1x1 convolution with batch norm (BN) [16] when the block changes the image size or the number of channels. Stages 2 and 3 halve the image size. A global average pool and one linear layer give the 10 class scores.

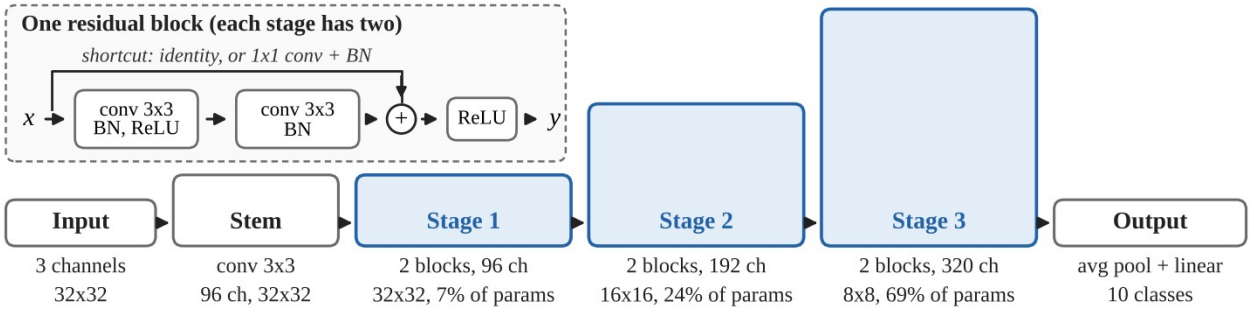


Figure 2. *The network (bottom) and one residual block (top). Box height shows the number of channels: the image gets smaller while the channels grow.*

Table 1. *Parameters of the final model.*

Part	Output size	Channels	Blocks	Parameters	Share
Stem: conv 3x3, BN, ReLU	32x32	96		2,784	0.1%
Stage 1	32x32	96	2	332,544	6.8%
Stage 2	16x16	192	2	1,181,568	24.1%
Stage 3	8x8	320	2	3,382,400	69.0%
Average pool + linear	1x1	10		3,210	0.1%
Total				**4,902,506**	**100%**

We chose this shape for four reasons:

- **Width beats depth at this budget.** A four stage ResNet-18 layout must shrink to 42 to 336 channels to fit and lost 1.8 points in our tests (Section 5). Three blocks per stage lost 0.4.
- **Most parameters sit in the last stage (69%).** Its 8x8 maps make it cheap to run.
- **1x1 shortcuts.** 3x3 shortcuts cost more parameters and scored 0.6 points lower.
- **Stable start.** The last batch norm in every block starts at zero, so at first each block only passes its input through the shortcut (Goyal et al. [12]). This keeps early training stable at a learning rate of 0.1.

3. Training Method

Table 2 lists the settings of our final model. All runs use the same base recipe. The baseline (wide3) trains for 200 epochs with label smoothing 0.1 and has no SAM, no CutMix and no teacher.

Table 2. *Training settings of the final model.*

Setting	Value
Data	45,000 images for training and 5,000 for validation (same split for every run); the final model is then retrained on all 50,000
Augmentation	random crop with 4 pixel padding, horizontal flip, AutoAugment with the CIFAR-10 policy [6], random erasing with p = 0.25 [8], CutMix on half of the batches [9]
Optimizer	SGD with Nesterov momentum 0.9, batch size 128, learning rate 0.1
Schedule	1,600 epochs: 5 warm-up epochs, then cosine decay to 0 [11]
Weight decay	5e-4 on weights only (not on batch norm or biases)
Label smoothing	none (0.1 in the baseline)
SAM	radius 0.05 [5]
Weight averaging	EMA with decay 0.999
Distillation	average of 5 teachers, weight $\alpha = 0.9$, temperature $T = 4$ (Section 4)
Precision	bfloat16 mixed precision
Test time	average of each image and its mirror image (flip TTA)

Augmentation. Each training image is changed at random every time it is used, so the model almost never sees the same picture twice: it is shifted by up to 4 pixels, flipped, changed in color, contrast or shape by AutoAugment, and one time in four has a rectangle erased. In half of the batches, CutMix also pastes a patch from another image and mixes the two labels in proportion to the patch size. This makes training harder, which is what lets the model train for 1,600 epochs without overfitting.

SAM. Sharpness aware minimization [5] looks for flat minima. Before each update it takes a small step uphill (radius 0.05) and uses the gradient from that point. This doubles the cost of each step. We used it because flat minima tend to cope better with small changes in the input, and the hidden test images look different from the training images.

Choosing the checkpoint. On the 45,000 image runs we keep the checkpoint (normal or EMA weights) with the best validation accuracy. The 50,000 image retrain has no validation set, so we submit its final EMA weights.

Following the rules. Our code refuses to write a submission from any model above 5,000,000 parameters, and it only accepts teachers trained from scratch by our own code. We never used the test images for training, for choosing models or for pseudo labels.

4. Knowledge Distillation

In knowledge distillation [3], a small student model learns from the predictions of bigger teacher models, not only from the labels. A teacher's output shows how much an image looks like each class (for example, a cat that looks a little like a dog), which tells the student more than the label alone. Our teachers are ResNets with 160, 320 and 640 channels and four blocks per stage (36,690,090 parameters each). We trained five of them from scratch on the same 45,000 training images, all with SAM and without CutMix (Table 3). They differ in seed, training length and augmentation, so that their mistakes partly cancel out in their average.

Table 3. *The five teachers. They are over the limit and only used for training.*

Teacher	Seed	Epochs	Augmentation	Val acc	Val acc (flip)
teacher	42	200	AutoAugment	97.28%	97.18%
teacher_s1	1	200	AutoAugment	97.16%	97.26%
teacher_s2	2	200	AutoAugment	97.38%	97.40%
teacher_s3	3	400	AutoAugment	97.74%	97.54%
teacher_s4	4	400	TrivialAugment [7]	97.68%	97.72%

During training, every batch goes through the five teachers and the student. The student's loss mixes the usual label loss with a term that pulls its predictions toward the teachers' average:

$$L = (1 - \alpha) \cdot CE(s, y) + \alpha \cdot T^2 \cdot KL(softmax(t / T) \,\|\, softmax(s / T))$$

Here s are the student's outputs, t is the average of the five teachers' outputs, y is the true label (a mix of two labels after CutMix), CE is cross entropy and KL is the Kullback-Leibler divergence. We use $\alpha = 0.9$, so most of the signal comes from the teachers, and $T = 4$, which softens the predictions so that the small class scores also carry information. The teachers see exactly the same augmented images as the student, so their soft labels always match the image (Beyer et al. [4]).

The students and the teachers do not use label smoothing: a teacher trained with it is worse for distillation (Müller et al. [10]), and the teachers' soft labels already give the student soft targets.

Does it help? With one teacher and 300 epochs, the student reaches 97.14%, against 96.86% for the same recipe without a teacher, a gain of about 0.3 points. Three teachers and 800 epochs gave 97.18%, and all five teachers with 1,600 epochs gave 97.46%. Our final student matches the average accuracy of its teachers (97.45%) with 7.5 times fewer parameters.

Cost. Distillation is not free, because every training step also runs the five teachers. One epoch of the student took about 23 seconds instead of 10 without a teacher, so the final 1,600 epoch run took about 10 hours.

5. Experiments

Short tests (60 epochs). To choose the design, we changed one thing at a time from the baseline recipe with 80, 160 and 320 channels, which reaches 95.34% after 60 epochs. With 5,000 validation images one standard error is about 0.3 points, so differences below about 0.6 points are noise (Figure 3).

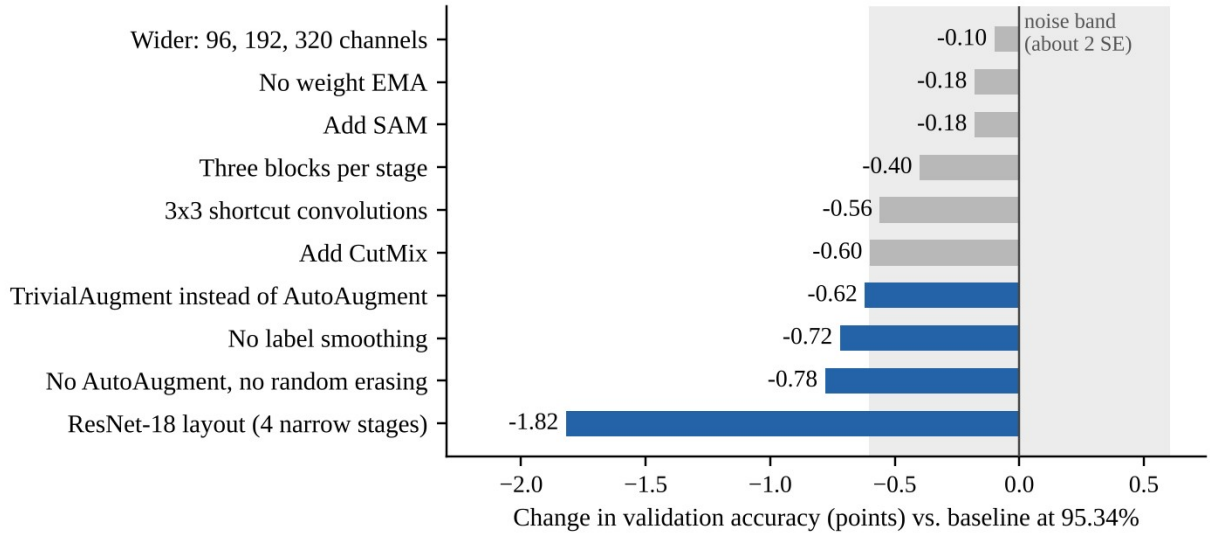


Figure 3. *Change in validation accuracy after 60 epochs (no TTA), compared with the baseline. Blue bars are outside the gray noise band.*

- **The layout matters most:** the ResNet-18 layout lost 1.8 points.
- **Augmentation and label smoothing help the baseline:** removing AutoAugment and random erasing cost 0.8 points, and removing label smoothing cost 0.7.
- **Some methods need long training:** SAM, CutMix and TrivialAugment looked worse after 60 epochs because they make training harder and slower at the start. Both SAM and CutMix helped in full length training in their original papers [5, 9], so we kept them for our long runs, but we could not afford a full length test of each one.

What we kept. From these tests we kept the three stage layout with two blocks per stage and 1x1 shortcuts, AutoAugment with random erasing, and EMA. We then widened the model to 96, 192 and 320 channels to use the full parameter budget. At 60 epochs it scored the same as the baseline within noise, and it is the base of all our students.

Test time augmentation (TTA). Averaging each image with its mirror image added 0.3 points for the baseline but almost nothing for the distilled models. Ten views with small shifts did not beat the mirror alone, so we submit with the mirror only.

6. Results

Table 4 lists our main runs, and Figure 4 shows where the final model makes mistakes. Validation accuracy comes from runs trained on 45,000 images. For the students, the Kaggle score comes from the same setup retrained on all 50,000 images. The baseline was submitted from its 45,000 image run, and student_no_kd is a control run that was not submitted.

Table 4. *Results of our main runs.*

Run	Params	Epochs	Teachers	Val acc	Val acc (flip)	Kaggle public
student_v5 (final)	**4.90M**	**1,600**	**5**	**97.46%**	**97.48%**	**0.899**
student_long	4.90M	800	3	97.18%	97.20%	0.896
student	4.90M	300	1	97.14%	97.12%	0.887
student_no_kd	4.90M	300	0	96.86%	96.88%	
wide3 (baseline)	4.34M	200	0	96.10%	96.44%	0.875

Validation and test scores differ by 7.6 to 8.9 points. The hidden test images are not the usual CIFAR-10 test set: they look brighter, they have more contrast, and none of them is a copy of a training image. Every model loses a similar amount on them, so the order of our models stayed the same. Recht et al. [13] saw the same effect: CIFAR-10 models lose 4 to 10 points on a newly collected test set but keep their ranking.

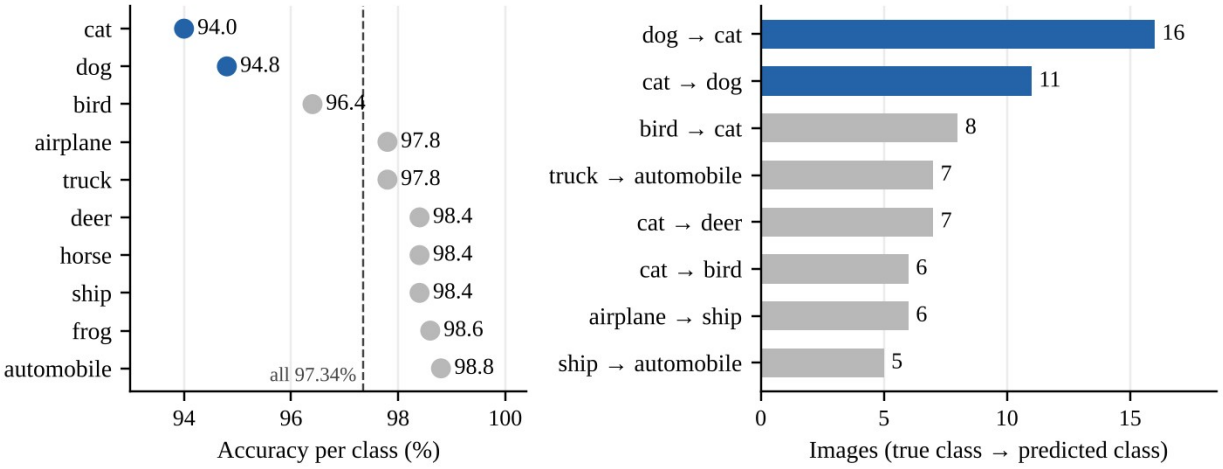


Figure 4. *Accuracy per class (left) and most common mistakes (right) of the final model.*

On the validation set (10 view TTA), cat (94.0%) and dog (94.8%) are the hardest classes. The most common mistakes are dogs called cats (16 of 500) and cats called dogs (11 of 500), which together make up 27 of the model's 133 errors. These two animals look alike at 32x32 pixels, and the teachers make the same mistakes, so distillation cannot fix them.

7. Discussion and Conclusion

What we learned. Short tests can mislead: SAM and CutMix slow down early training, so a 60 epoch test can make them look worse than they are in a long run. Noise also limits what we can see: one run on 5,000 validation images has about 0.3 points of noise and the 1,000 public test images about 1 point, so we chose models by validation accuracy, not by the leaderboard. Finally, Table 5 shows that distillation adds about 0.3 points, while the rest of our gain comes from the wider model, SAM, CutMix, more teachers and longer training.

Table 5. *Where our gain came from (validation accuracy, gain in points).*

Run	Change from the row above	Val acc	Gain
wide3 (baseline)	80, 160, 320 channels, 200 epochs	96.10%	
student_no_kd	96, 192, 320 channels, SAM, CutMix, no label smoothing, 300 epochs	96.86%	+0.76
student	one teacher	97.14%	+0.28
student_long	three teachers, 800 epochs	97.18%	+0.04
student_v5	**five teachers, 1,600 epochs**	**97.46%**	**+0.28**

Limitations. We trained each setting once, with one seed, and could not afford full length tests of SAM, CutMix and the number of teachers one by one, so we cannot say exactly how much each one adds. The hidden test images are brighter and have more contrast than our training images, and the rules do not allow training on them, so we relied on robust training (strong augmentation, SAM and several teachers) instead.

Next steps. We would test each method at full length with two or three seeds, try brightness and contrast augmentation for the shift in the test images, and check whether fewer, longer trained teachers give the same gain at a lower cost, since every training step runs all five teachers.

Reproducibility. All code (written in PyTorch [17]), configs, training logs and the notebook are in the repository (link on page 1). The notebook rebuilds every table and figure from the saved logs without a GPU, and automated tests check the competition rules: training never reads the test images, the teachers never saw the validation images, and no model above 5,000,000 parameters can be submitted.

Conclusion. A ResNet with fewer than 5 million parameters can match much larger models on CIFAR-10. Our final model uses three wide stages, strong augmentation, SAM and weight averaging, and learns from five teachers that each have 7.5 times more parameters. It reaches 97.46% on our validation set, the same as the teachers' average, and 0.899 on the Kaggle public leaderboard. The main open problem is the gap to the hidden test images, which look different from the training data.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR 2016.

[2] S. Zagoruyko, N. Komodakis. Wide Residual Networks. BMVC 2016.

[3] G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. 2015.

[4] L. Beyer et al. Knowledge Distillation: A Good Teacher is Patient and Consistent. CVPR 2022.

[5] P. Foret, A. Kleiner, H. Mobahi, B. Neyshabur. Sharpness-Aware Minimization for Efficiently Improving Generalization. ICLR 2021. SAM code adapted from github.com/davda54/sam (MIT license).

[6] E. D. Cubuk et al. AutoAugment: Learning Augmentation Policies from Data. CVPR 2019.

[7] S. G. Müller, F. Hutter. TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation. ICCV 2021.

[8] Z. Zhong et al. Random Erasing Data Augmentation. AAAI 2020.

[9] S. Yun et al. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. ICCV 2019.

[10] R. Müller, S. Kornblith, G. Hinton. When Does Label Smoothing Help? NeurIPS 2019.

[11] I. Loshchilov, F. Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. ICLR 2017.

[12] P. Goyal et al. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. 2017.

[13] B. Recht, R. Roelofs, L. Schmidt, V. Shankar. Do CIFAR-10 Classifiers Generalize to CIFAR-10? 2018.

[14] K. Liu. pytorch-cifar, reference CIFAR-10 ResNet code. github.com/kuangliu/pytorch-cifar.

[15] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009.

[16] S. Ioffe, C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML 2015.

[17] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.

AI use. Parts of the code and of this report were written with help from an AI assistant (Claude), which the course allows when it is acknowledged. The authors checked all experiments and results.

Acknowledgements. We thank NYU IT High Performance Computing for the GPU time.